

Preference Post-Training Qwen2.5-0.5B

Supervised Fine-Tuning, Reward Modeling, and Guarded PPO on Diverse, Long-Form HelpSteer3 Data

Dheeraj Dhillon
July 2026

Abstract

This work asks whether a complete reinforcement learning from human feedback (RLHF) pipeline can remain measurable and inspectable under notebook-scale compute without reducing preference learning to short or single-domain outputs. Experiments use QWEN2.5-0.5B-INSTRUCT and NVIDIA’s heterogeneous HELPSTEER3-PREFERENCE. A sequence-length audit motivates a 4,096-token budget for supervised fine-tuning (SFT) and reward modeling, reducing observed truncation from about 40% to about 1%. One epoch of response-only LoRA SFT supplies a common policy and reference; a two-epoch Bradley–Terry reward model then reaches 65.62% held-out pairwise accuracy.

The final policy is trained with guarded, KL-regularized PPO for 12,032 on-policy rollouts and 188 optimizer updates. The run combines domain-balanced sampling, quantile-calibrated reward clipping, EOS-aware terminal scoring, smooth repeated-4-gram shaping, exact resume, and memory-efficient response-only log-probability computation for 768-token generations. On all 2,017 validation prompts, PPO remains numerically stable and avoids empty-output collapse, but the learned reward model prefers it over Base in 817 of 2,017 comparisons (40.51%) and over SFT in 974 (48.29%). PPO nevertheless remains close to the baselines in length and stopping behavior: its median response is 355 tokens, its cap-hit rate is 16.31%, and 16.16% of responses cross a 25% repeated 4-gram threshold. The result is therefore presented as an end-to-end systems, stability, and evaluation study—not as evidence that PPO universally improves the base instruction model. Complete generations and audit metadata are made available for direct inspection.

1 Introduction

RLHF connects next-token language modeling with human judgments of helpfulness, relevance, correctness, clarity, and safety. A canonical pipeline first adapts a policy to preferred demonstrations, learns a scalar reward from response comparisons, and then optimizes the policy while constraining drift from a supervised reference [?, ?].

This project deliberately pairs a modest policy and compute budget with a nontrivial preference distribution. HELPSTEER3-PREFERENCE includes general, code, STEM, and multilingual tasks, long conversational contexts, executable snippets, and multi-step explanations. The small model makes iteration feasible, but it also makes sequence truncation, reward-model error, stopping behavior, and value estimation consequential. The Qwen2.5 family is described in [?].

The work began with custom SFT, reward-model, and token-level PPO loops, then migrated trainer mechanics to Hugging Face TRL while retaining repository- owned preprocessing, model initialization, exact-resume state, reward calibration, evaluation, and qualitative auditing. The central contribution is not a state-of-the-art aligned model. It is an inspectable pipeline in which training behavior and failure analysis are treated as equally important outputs.

Why SFT an instruction model? The starting checkpoint is already instruction-tuned. Continued SFT is used as domain and preference adaptation, not instruction tuning from scratch. It moves the policy toward the preferred HelpSteer3 response distribution and creates a reproducible checkpoint shared by reward modeling, the PPO policy, and the frozen PPO reference. This controlled initialization makes downstream changes easier

to attribute.

2 Method

2.1 Data and sequence handling

HELPSTEER3-PREFERENCE contains 40,476 pairwise records over general, code, STEM, and multilingual tasks [?]. Each record contains one conversational context, two candidate responses, an overall preference in $\{-3, \dots, 3\}$, and up to three annotator judgments. Tied or invalid pairs are excluded, leaving 36,264 training and 1,917 reward-model validation pairs. The generation suite retains all 2,017 validation prompts.

All stages use the Qwen chat template and distinct PAD and EOS tokens. Completions are preserved through EOS; when an example exceeds 4,096 tokens, old prompt turns are removed before a final prompt-side token fallback. At a 1,024-token budget, 38.47% of SFT examples and 40.82% of reward-model pairs would truncate. At 4,096 tokens, the observed rates fall to 0.83% and 1.00%. Sequence handling is therefore part of the experiment rather than a cosmetic configuration choice.

2.2 Supervised fine-tuning

For prompt x , preferred response $y = (y_1, \dots, y_T)$, and completion mask m_t , response-only SFT minimizes

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T m_t \log \pi_{\theta}(y_t \mid x, y_{<t}). \quad (1)$$

Prompt and padding tokens do not contribute to the loss. LoRA [?] rank-16 adapters are trained in attention and feed-forward projections while the backbone remains frozen. The merged model supplies the SFT comparison

policy and the common initialization/reference for later stages.

2.3 Reward modeling

The reward model begins from the merged SFT backbone and adds a scalar head. For preferred y^+ and rejected y^- under the same prompt, it minimizes the Bradley–Terry loss

$$\mathcal{L}_{\text{RM}}(\phi) = -\log \sigma(r_\phi(x, y^+) - r_\phi(x, y^-)). \quad (2)$$

Only score differences are identifiable; the sign of one isolated score is not a universal quality boundary. A persisted offset estimated on preferred demonstrations is subtracted consistently during PPO and evaluation.

2.4 Guarded KL-regularized PPO

PPO [?] receives a terminal reward-model score and token-level KL shaping against the frozen SFT policy:

$$r_t^{\text{KL}} = -\beta [\log \pi_\theta(y_t | s_t) - \log \pi_{\text{ref}}(y_t | s_t)], \quad (3)$$

$$R(x, y) = \tilde{r}_\phi(x, y) + \sum_t r_t^{\text{KL}}. \quad (4)$$

With generalized advantages $\hat{A}_t$ and $\rho_t = \pi_\theta(y_t | s_t) / \pi_{\text{old}}(y_t | s_t)$, the policy loss is

$$\mathcal{L}_{\text{PPO}} = -\mathbb{E} \left[\min \{ \rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \} \right]. \quad (5)$$

The final path follows high-impact implementation details from the N+ RLHF/PPO reproduction [?]: zero policy dropout, behavior probabilities matched to sampling temperature, a frozen SFT reference, an SFT-initialized reward model, controlled scalar-head initialization, an RM-initialized critic, masked advantage whitening, Adam $\epsilon = 10^{-5}$, and fixed-length sampling followed by EOS truncation.

The guarded run adds controls motivated by earlier failures. Reward bounds are calibrated from 4,096 domain-stratified reward-training pairs. Raw terminal scores are clipped to the 0.5th and 99.5th percentiles, $[-2.7507, 1.5018]$; a missing EOS receives the same lower-bound score. A smooth penalty is applied only when repeated token 4-gram mass exceeds the 95th percentile of preferred responses (0.3084). Reward whitening is disabled so these calibrated penalties retain their scale, while TRL’s masked advantage whitening remains active. Every batch of 64 contains exactly 16 prompts from each domain, preventing general prompts from dominating the on-policy data.

2.5 Memory-efficient long-response execution

Long-form PPO exposed systems costs that are absent from short demonstrations. The stock rollout path retained full-vocabulary generation scores, implying a $64 \times 768 \times 151,665$ tensor, or about 27.77 GiB in float32, before the optimizer update. The final path generates token IDs with a rollout-only KV cache and no generation-score history, then recomputes behavior log-probabilities

in chunks from response-position logits only. It removes prompt columns masked for every row in each chunk, shares the policy backbone with the reference by disabling the LoRA adapter, and backpropagates policy and value losses through separate graphs before one accumulated optimizer step. These transformations preserve rewards, GAE, clipping, effective batch size, and sampled trajectories while reducing peak allocated memory to 37.64 GiB on the completed run.

3 Training Results

3.1 SFT and reward model

The one-epoch SFT run is stable. Validation loss changes only from 1.1183 at step 250 to 1.1127 at step 1,134, while completion-token accuracy remains near 72%. These metrics do not prove better open-ended responses; they establish a stable HelpSteer-adapted policy and common reference.

The two-epoch reward model reaches 65.62% overall pairwise accuracy. Accuracy increases with preference strength: 61.23%, 66.33%, and 71.10% for strengths 1, 2, and 3. Domain accuracy is uneven: 70.23% on code, 62.91% on general, 59.26% on STEM, and 71.82% on multilingual examples. It is therefore usable as a noisy preference signal, but not reliable enough to serve as ground truth, especially for correctness-sensitive STEM outputs.

3.2 PPO dynamics

The guarded run completed its configured budget: 12,032 rollouts and 188 updates. The final response-level objective KL to the frozen SFT reference is 0.7305, or 0.00199 per response token. The old-to-new optimizer update remains much smaller: approximate KL is 0.000537 and clip fraction is 0.00571 at update 188. Final value loss is 0.1076, down from the large early values produced while the critic adapted to the terminal reward scale. The final rollout batch has 82.81% EOS completion, 17.19% cap hits, mean response length 366.8 tokens, and no empty responses.

Across the run, later batches are shorter, more likely to terminate, and easier for the critic to predict than early batches. This is evidence that the guarded training mechanics worked. It is not evidence that the learned reward objective became an independent measure of human quality.

4 Evaluation

4.1 Protocol

Base, SFT, and PPO each generate one response for all 2,017 validation prompts using the same tokenizer, prompt formatting, 3,072-token prompt limit, 768-token response cap, temperature 0.7, and top- $p = 1$. The same epoch-two reward model scores all prompt-response pairs. Generation is sequential and resumable at the policy-output level; all pairwise comparisons derive from one shared response table. Because the PPO training judge

Table 1: Final training profile. “Token accuracy” is teacher-forced next-token accuracy over preferred completion tokens, not task accuracy.

Stage	Data and budget	Adaptation	Optimization	Outcome
SFT	36,264 train; 1,917 val.; 4,096 total tokens	LoRA $r = 16$, $\alpha = 32$, dropout 0	1 epoch; effective batch 32; LR 5×10^{-6}	Train loss 1.0556; val. loss 1.1127; token accuracy 72.02%
Reward model	36,264 pairs; 1,917 val. pairs; 4,096 tokens	SFT backbone; scalar head; LoRA $r = 32$, $\alpha = 64$	2 epochs; effective batch 64; LR 5×10^{-6} ; centering 0.01	Val. loss 0.6166; pairwise accuracy 65.62%; mean margin 0.3578
Guarded PPO	12,032 rollouts; 12,007 unique prompts; 768-token responses	SFT policy/reference; RM-initialized value model	188 updates; 64 rollouts/update; 4 PPO epochs; LR 3×10^{-6} ; $\beta = 0.10$	Final objective KL 0.7305; update KL 0.000537; EOS 82.81%; no empty collapse

is also the evaluator, learned reward is treated as a diagnostic rather than independent human preference.

The learned-reward differences are small in the mean, but the win counts favor Base. PPO is closest to SFT and exceeds SFT on the STEM subset, yet it does not beat Base in any of the four domains. This is a mixed alignment outcome rather than a proxy-judge victory.

4.2 Stopping, repetition, and qualitative audit

The guarded PPO response distribution is much closer to Base and SFT than the earlier, more exploitable PPO checkpoint archived in the technical companion. In the final suite, 16.16% of PPO responses cross the 25% repeated-4-gram threshold and 5.21% cross 50%, compared with 8.18% and 2.13% for Base and 18.54% and 7.09% for SFT. PPO’s 16.31% cap-hit rate is nearly identical to Base’s 16.21% and below SFT’s 18.34%.

The automated audit produces review queues rather than semantic verdicts: 9 qualified PPO candidates, 120 strong PPO-loss candidates, 326 repetition-risk rows, and 155 high-reward repetition mismatches. These sets overlap and are intended to direct manual inspection. Positive cases include clearer structure, better coverage of multi-part requests, and more compact answers. Negative cases include incorrect code, confused factual reasoning, prompt restatement, and occasional extreme loops that still receive high reward. The response explorer publishes the full prompt, Base and PPO generations, reward scores, token counts, EOS/cap state, and repeated 4-gram diagnostics so that these claims can be checked against concrete text.

5 Discussion

The strongest supported result is a systems-and-evaluation result. The project connects long-context preprocessing, parameter-efficient SFT, pairwise reward modeling, guarded online PPO, exact-resume execution, full-set generation, and transparent qualitative audit on a diverse preference dataset. The final PPO run is numerically healthy and materially less prone to the stopping and repetition failures that motivated its guardrails. It does not, however, improve aggregate

learned-reward preference over the already instruction-tuned Base model.

This distinction is central. Stable clipping, low update KL, finite value loss, and high EOS rate show that the optimizer behaves as intended; they do not establish that the scalar objective captures every property a human reviewer cares about. Reward calibration and repetition shaping reduce obvious proxy exploitation, but they cannot make a 65.62%-accurate reward model an oracle. The project therefore treats reward scores, behavioral diagnostics, and direct response inspection as complementary evidence.

The central limitations are that the same reward model supplies both training and quantitative evaluation; no blinded multi-rater human study is reported; one sampled generation per policy does not estimate decoding variance; length remains entangled with completeness, formatting, and repetition; and a 0.5B policy/reward model has limited factual and long-context capacity. The final checkpoint completed a predefined 12,000-episode budget rather than being selected by an independent human metric. These constraints bound the result, but they do not invalidate the completed pipeline or the stability analysis.

Future work should prioritize independent evaluation over more PPO updates: blinded human or strong-judge comparisons on a stratified subset, paired confidence intervals across repeated generations, reward-model hard negatives from observed failures, execution-based checks for code, stronger STEM correctness tests, and multi-metric checkpoint selection. Scaling the policy is most useful after the evaluator is less exploitable.

6 Conclusion

One-epoch continued SFT provides a stable HelpSteer-adapted reference, the reward model learns a meaningful but imperfect ranking signal, and the guarded PPO run completes 188 updates over 12,032 long-form rollouts without numerical or empty-output collapse. Quantile reward bounds, exact-EOS scoring, KL regularization, repetition shaping, domain balance, exact resume, and response-only memory optimizations make this the most controlled PPO iteration in the project. Full evaluation does not show aggregate reward-model superiority over

Table 2: Full policy-suite evaluation on 2,017 prompts. Repetition is the fraction of responses whose repeated word-level 4-gram mass exceeds 25%. The behavioral diagnostics are placed before reward/win statistics intentionally.

Policy	Mean tok.	Median tok.	Cap hit	Rep-4 > 25%	Mean reward	Three-way wins	Win rate	Empty
Base	373.5	333	16.21%	165 (8.18%)	0.2229	845	41.89%	0.00%
SFT	392.5	364	18.34%	374 (18.54%)	0.2037	585	29.00%	0.00%
PPO	384.1	355	16.31%	326 (16.16%)	0.1753	563	27.91%	0.00%
Tie	—	—	—	—	—	24	1.19%	—

Table 3: Pairwise learned-reward results. Win rates include ties in the denominator.

Comparison	Left wins	Right wins	Ties	Right win rate	Mean right-left delta
Base vs SFT	1,145	849	23	42.09%	−0.0192
Base vs PPO	1,179	817	21	40.51%	−0.0476
SFT vs PPO	1,018	974	25	48.29%	−0.0284

Base, but it shows a policy whose stopping and repetition behavior remains usable enough for meaningful side-by-side inspection. The appropriate conclusion is neither that PPO solved alignment nor that the training did nothing: the project demonstrates how to build, stabilize, audit, and honestly report an end-to-end RLHF system under constrained compute.

References

- [1] N. Stiennon et al. Learning to summarize with human feedback. *NeurIPS*, 2020. <https://arxiv.org/abs/2009.01325>.
- [2] L. Ouyang et al. Training language models to follow instructions with human feedback. *NeurIPS*, 2022. <https://arxiv.org/abs/2203.02155>.
- [3] S. Huang, M. Noukhovitch, A. Hosseini, K. Rasul, W. Wang, and L. Tunstall. The N+ Implementation Details of RLHF with PPO. arXiv:2403.17031, 2024. <https://arxiv.org/abs/2403.17031>.
- [4] Z. Wang et al. HelpSteer3-Preference: Open Human-Annotated Preference Data across Diverse Tasks and Languages. arXiv:2505.11475, 2025. <https://arxiv.org/abs/2505.11475>.
- [5] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. *ICLR*, 2022. <https://arxiv.org/abs/2106.09685>.
- [6] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>.
- [7] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024. <https://arxiv.org/abs/2412.15115>.
- [8] L. von Werra et al. TRL: Transformer Reinforcement Learning. Hugging Face, 2020–2026. <https://huggingface.co/docs/trl/>.